/* Source: http://www.ypologist.com/mmalita17 */

/* File: Puzzles/zebra.pro
Title: Finding the zebra

There are 15 facts and 2 questions:
Who has a zebra and who drinks water?

1) There are 5 colored houses in a row, each with an owner, an animal, a cigarette, a drink.
2) The English lives in the red house.
3) The Spanish has a dog.
4) They drink coffee in the green house.
5) The Ukrainian drinks tea.
6) The green house is next to the white house.
7) The Winston smoker has a serpent.
8) In the yellow house they smoke Kool.
9) In the middle house they drink milk.
10) The Norwegian lives in the first house from the left.
11) The Chesterfield smoker lives near the man with the fox.
12) In the house near the house with the horse they smoke Kool.
13) The Lucky Strike smoker drinks juice.
14) The Japanese smokes Kent.
15) The Norwegian lives near the blue house.

Uses: SWI: member/2,length/2

We represent the houses as a list with 5 lists from left to right:

Sol = [[Man, Animal, Cigarette, Drink, Color], [..],[..],[..],[..]]

```
?- start(S),write_list(S),fail.
[norwegian,fox,kool,water,yellow]
[ukrainian,horse,chesterfield,tea,blue]
[english,snake,winston,milk,red]
[japonese,zebra,kent,coffee,green]
[spanish,dog,lucky,juice,white]
no
*/
near(X,Y,L):-nextto(X,Y,L) ; nextto(Y,X,L).

start(Sol) :- length(Sol,5),                % 1
    member([english,_,_,_,red],Sol),        % 2
    member([spanish,dog,_,_,_],Sol),        % 3
    member([_,_,_,coffee,green],Sol),       % 4
    member([ukrainian,_,_,tea,_],Sol),      % 5
    near([_,_,_,_,white],[_,_,_,_,green], Sol), % 6
    member([_,snake,winston,_,_],Sol),      % 7
    member([_,_,kool,_,yellow],Sol),        % 8
    Sol= [_,_,[_,_,_,milk,_],_,_],          % 9
    Sol= [[norwegian,_,_,_,_],_,_,_,_],     % 10
    near([_,_,chesterfield,_,_],[_,fox,_,_,_],Sol), % 11
    near([_,_,kool,_,_],[_,horse,_,_,_], Sol), % 12
    member([_,_,lucky,juice,_],Sol),            % 13
    member([japonese,_,kent,_,_],Sol),          % 14
    near([norwegian,_,_,_,_],[_,_,_,_,blue],Sol),  % 15
    member([_,_,_,water,_],Sol),        % someone drinks water
    member([_,zebra,_,_,_],Sol).        % someone has a zebra
```

```
/* File: Puzzles/beds1.pro    Author: sol MMalita
Title: Bunk beds
From:  http://eduplace.com/math/brain/65-294.html
All 6 girls in the junior cabin at camp sleep in bunk beds.
Each girl has a different color blanket on her bed.
The beds are lined up in a row with red, black, yellow, brown, blue and green blankets.
1. Reeva and Haley is closest to the door.
2. The color of the blankets on the bottom bunks all begin with the same letter
3. The color of Beth's blanket doesn't begin with the same letter as the beginning of her
name
4. The blue blanket is under the red blanket.
5. Liza and Kayla like their lower bunks.
6. Kayla is in the middle bed.
7. Zoe doesn't share a bunk with Kayla.
8. The brown blanket is at the end of a row.
9. The yellow is above the black blanket.
Sol= [[Name,Color],[Name,Color],...]
They are in the following order bottom, up, bottom, up
from left to right
?- start(I).
I = [[reeva,black],[haley,yellow],[kayla,blue],[beth,red],[liza,brown],[zoe,green]] ;
I = [[reeva,blue],[haley,red],[kayla,black],[beth,yellow],[liza,brown],[zoe,green]] ;
I = [[haley,black],[reeva,yellow],[kayla,blue],[beth,red],[liza,brown],[zoe,green]] ;
I = [[haley,blue],[reeva,red],[kayla,black],[beth,yellow],[liza,brown],[zoe,green]] ;
*****************************************************/
start(Sol):- Sol=[[N1,C1],[N2,C2],[kayla,C3],[N4,C4],[N5,C5],[N6,C6]],
        ((N1=reeva,N2=haley);(N2=reeva,N1=haley)),  /* 1.*/
        member(C1,[black,blue,brown]), /* 2 */
        member(C3,[black,blue,brown]),
        member(C5,[black,blue,brown]),
        member([blue,red],[[C1,C2],[C3,C4],[C5,C6]]), /* 4 */
```

```
member(liza,[N1,N5]),  /* 5 */
member(zoe,[N1,N2,N4,N5,N6]),
brown=C5,              /* 8 */
member([black,yellow],[[C1,C2],[C3,C4],[C5,C6]]), /* 9 */
member(green,[C1,C2,C3,C4,C5,C6]),
member([beth,C],Sol),member(C,[red,yellow,green]),   /* 3 */
not(N4=zoe). /* 7 */
```

```
/* File: Puzzles/jos1.pro   Author: (Sol MM)
   Title: Finding the profession
Smith, Baker, Carpenter and Tailor have each a profession
(smith, baker, carpenter, tailor) but not showed by their names.
Each of them has a son.
But the sons also do not have the profession showed by their name.
If you know that:
1) no son has the same profession as his father has
and
2) Baker has the same profession as Carpenter's son has
and
3) Smith's son is a baker.
find the professions of the parents and sons.
*/
professions([smith,baker,carpenter,tailor]). % list of professions
/* S = Smith's profession              B = Baker's profession
   C = Carpenter's profession          T = Taylor's profession
   Son_S = Smith son's profession      Son_B = Baker son's profession
   Son_C = Carpenter son's profession  Son_T = Taylor son's profession
*/
sol([S,B,C,T],[Son_S,Son_B,Son_C,Son_T]):-
    professions(L),
    member(S,L),not(S)=smith,
    member(B,L),not(B)=baker,
    member(C,L),not(C)=carpenter,
    member(T,L),not(T)=taylor,
/* The sons do not have the same profession as their name shows */
    member(Son_S,L),not(Son_S)=smith,
    member(Son_B,L),not(Son_B)=baker,
    member(Son_C,L),not(Son_C)=carpenter,
    member(Son_T,L),not(Son_T)=taylor,
```

```
/* The sons do not have the same profession as their fathers either */
    not(S) = Son_F,
    not(B) = Son_B,
    not(C) = Son_T,
    not(T) = Son_C,
    B=Son_C,          % Baker has the same profession as Carpenter's son.
    Son_S=baker.      % Smith's son is a baker
```

```
/* File: Puzzles/lunch2.pro   Author: solution MMalita
   Title: Insider Trading
   From:  Dell- Math Puzzles Logic Problems, Jan 2002, p 36.
Kids are notorious for trading their lunches away, and the kids at
PT Elementary are no exception. Mrs. Day, the lunch lady recently watched four
big-time traders at work, swapping everything their parents had packed.
She had trouble keeping it all straight, but shrugged when she realized
that May and her friends each finished with a sandwich, piece of fruit,
and a dessert (one was a brownie).
Can you untangle the " insider trading" and determine what each child
ate and who brought it to school?
1. Each child swapped with each friend exactly once,
   always trading like items
2. The child who ate the apple also gobbled up the chocolate cupcake.
3. Jay traded his fruit for Ray's orange.
4. One child bought a bologna sandwich and a banana to school.
5. The child who ate the turkey sandwich also ate the grapes.
6. The child who ate the ham cheese sandwich traded away the cookie
    Mom had packed.
7. Ray enjoyed Faye's peanut butter sandwich.
8. The child who ate Joy's sandwich also ate the candy bar.
| ?- start(S).
[may,bologna,turkey,banana,grapes,cupcake,candy_bar]
[jay,turkey,bologna,apple,orange,brownie,cookie]
[ray,ham_cheese,peanut,orange,apple,candy_bar,cupcake]
[faye,peanut,ham_cheese,grapes,banana,cookie,brownie]
*/
/*  1) capture the constants and classify them
    2) Variable space: B= Brought     E = Eaten
sol([[Child,Sandwich_B,Sandwich_E,Fruit_B,Fruit_E,Dessert_B,Dessert_E],
     [Child,Sandwich_B,Sandwich_E,Fruit_B,Fruit_E,Dessert_B,Dessert_E],
```

```
     [Child,Sandwich_B,Sandwich_E,Fruit_B,Fruit_E,Dessert_B,Dessert_E],
     [Child,Sandwich_B,Sandwich_E,Fruit_B,Fruit_E,Dessert_B,Dessert_E],
     [Child,Sandwich_B,Sandwich_E,Fruit_B,Fruit_E,Dessert_B,Dessert_E]]).
Order is not important in this example. that is why we write
Child= [C1,C2,C3,C4], otherwise we get all combinations.
*/

start(Sol):- Dessert=[candy_bar,cupcake,cookie,brownie],
             Fruit=[grapes,orange,apple,banana],
             Sand=[turkey,peanut,ham_cheese,bologna],
             Child=[may,jay,ray,faye],Child=[C1,C2,C3,C4],

Sol=[[C1,Sb1,Se1,Fb1,Fe1,Db1,De1],[C2,Sb2,Se2,Fb2,Fe2,Db2,De2],[C3,Sb3,Se3,Fb3,Fe3,Db3,De3]
,
          [C4,Sb4,Se4,Fb4,Fe4,Db4,De4]],
%3. Jay traded his fruit for Ray's orange.
    member([jay,Sbjay,_,Fbjay,orange,_,_],Sol),
%8. The child who ate Jay's sandwich also ate the candy bar.
    member([_,_,Sbjay,_,_,_,candy_bar],Sol),
%5. The child who ate the turkey sandwich also ate the grapes.
    member([_,_,turkey,_,grapes,_,_],Sol),
%7. Ray enjoyed Faye's peanut butter sandwich.
    member([ray,Sbfaye,peanut,orange,Fbjay,_,_],Sol),
    member([faye,peanut,Sbfaye,_,_,_,_],Sol),
%2. The child who ate the apple also gobbled up the chocolate cupcake
    member([_,_,_,_,apple,_,cupcake],Sol),
%4. One child bought a bologna sandwich and a banana to school
    member([_,bologna,_,banana,_,_,_],Sol),
%6. The child who ate the ham cheese sandwich traded away the cookie
    member([_,_,ham_cheese,_,_,cookie,_],Sol),
%1. Each child swapped with each friend exactly once, always trading like items
```

```
    permutation([Sb1,Sb2,Sb3,Sb4],Sand),
    permutation([Se1,Se2,Se3,Se4],Sand),
    permutation([Fb1,Fb2,Fb3,Fb4],Fruit),
    permutation([Fe1,Fe2,Fe3,Fe4],Fruit),
    permutation([Db1,Db2,Db3,Db4],Dessert),
    permutation([De1,De2,De3,De4],Dessert),
    list_comb(2,Sol,Pairs),all_prop(intersection_2,Pairs).

/* %1. Each child swapped with each friend exactly once,always trading like items
All the pairs have intersection_2 means 2 items in common
Take all the pairs (only once) and see if they have exactly
2 elements in common:
*/
intersection_2([X,Y]):-intersection(X,Y,R),length(R,2).
/* mem1(Lr,L). For comb/3. Same as mem/2 but does not generate [a,b] and [b,a].
    ?- mem1([X,Y],[a,b,c]).
    [a,b][a,c][b,c]
*/
mem1([],Y).
mem1([H|T],Y):-member(H,Y),rest(H,Y,New),mem1(T,New).

rest(A,L,R):-Y=[A|R],append(X,Y,L),!.
/* comb(N,L,Res). Combinations. Arrangements without " order".
    | ?- comb(2,[a,b,c],I).
    I = [a,b] ;  I = [a,c] ;  I = [b,c] ;
*/
comb(N,L,X):-length(X,N),mem1(X,L).
/* list_comb(N,L,Res).
    ?-  list_comb(2,[a,b,c,d],L).
    L = [[a,b],[a,c],[a,d],[b,c],[b,d],[c,d]]
*/
```

```
list_comb(N,L,Res):- findall(X,comb(N,L,X),Res).
/* all_prop(P,L). All elements from a list (first level) have property Prop.
EX: check if all are integers in a list
?- P=integer,L=[1,2,3],forall(member(X,L),(F=..[P,X],call(F))).
P = integer,L = [1, 2, 3].
*/
all_prop(P,L):-forall(member(X,L),(F=..[P,X],call(F))).
```

```
/* File: Puzzles/costume.pro  Solution MMalita
From: Dell- Math Puzzles Logic Problems,   Jan 2002 p.15
Title: The costume ball
1    Four couples in all
     Attended a costume ball.
2    The lady dressed as a cat
     Arrived with her husband Matt.
3    Two couples were already there,
     One man dressed like a bear.
4    First to arrive wasn't Vince,
     But he got there before the Prince.
5    The witch (not Sue) is married to Chuck,
     Who was dressed as Donald Duck.
6    Mary came in after Lou,
     Both were there before Sue.
7    The Gipsy arrived before Ann,
     Neither is wed to Batman.
8    If Snow White arrived after Tess,
     Then how was each couple dressed?
?- sol(I),write(I),fail.
I= [[lou,bear,tess,gipsy],[vince,batman,mary,snow_white],[matt,prince,sue,cat],
     [chuck,donald_duck,ann,witch]] %%(There is only one solution.)
                            second     third        last
[[Man,Costume_man,Woman,Costume_woman],[_,_,_,_],[_,_,_,_],[_,_,_,_]]
**************************************************/
boy([vince,chuck,lou,matt]).
girl([sue,mary,ann,sue]).
costume_boy([batman,donald_duck,prince,bear]).
costume_girl([witch,gipsy,cat,snow_white]).

start(Sol):-  boy(B),girl(W),costume_boy(C),costume_girl(Cw),
```

```
Sol=[[M1,C1,W1,Cw1],[M2,C2,W2,Cw2],[M3,C3,W3,Cw3],[M4,C4,W4,Cw4]], % 1
member([matt,_,_,cat],Sol),                          % 2
Sol=[X1,X2,[matt,_,_,cat],_],                  % 3
 (member(bear,X1) ; member(bear,X2)),                % 3
before([vince,_,_,_],[_,prince,_,_],Sol),     % 4
member([chuck,donald_duck,X,witch],Sol),      % 5
before([lou,_,_,_],[_,_,mary,_],Sol),         % 6
before([_,_,mary,_],[_,_,sue,_],Sol),         % 6
before([_,G1,_,gipsy],[_,A1,ann,_],Sol),           % 7
before([_,_,tess,_],[_,_,_,snow_white],Sol), % 8
permutation([M1,M2,M3,M4],B),
permutation([C1,C2,C3,C4],C),
permutation([W1,W2,W3,W4],G),
permutation([Cw1,Cw2,Cw3,Cw4],Cw),
vince \== M1,                     % 4
X\== sue,                         % 5
G1 \== batman, A1 \== batman.   % 7
```

```
/* File: Puzzles/aa6.pro Author: (sol MM)
Title: Thursday Party
From: Adrian Atanasiu http://www.be-logic.ro
Four ladies meet each week on Thursday to play bridge.
On each meeting they decide what everyone has to bring for the next meeting.
1. Mrs. Andrew will  bring chocolate cake.
2. Neither Mrs. Brown, nor Vivien, nor Ann Clark will bring cookies.
3. Rachel, which is not from Davidson's family, will bring coffee.
4. Mary will not bring the wine.
Find the whole name of each lady and what is she supposed to bring next week?
Final = [[ LastName, FirstName, Bring]...]
Solutions found with the program: ?- start(I).
I = [[clark,ann,wine],[davidson,mary,cookie],
[brown,rachel,cofee],[andrew,vivien,chocolate]] ;
*********************************************************/
start(Final):- Final=[[_,_,wine],[_,_,cookie],[_,_,cofee],[_,_,_]],
             member([andrew,_,chocolate],Final),
             member([brown,_,Bb],Final), Bb \== cookie),
             member([_,vivien,Bv],Final),Bv \== cookie,
            member([clark,ann,Ba],Final), Ba \== cookie,
            member([_,mary,Bm],Final), Bm \== wine,
            member([davidson,_,_],Final),
            member([X,rachel,cofee],Final), X \== davidson.
```

```
/*   File: Puzzles/aa1.pro     Author: (sol MM)
     Title: Contest: swimming and biking.
     From: Adrian Atanasiu: www.be-logic.ro
Four students competed in two different tests: swimming and biking.
Can you establish the final order for each test?
1. Andrew did not win any competition.
2. The person who won the swimming contest was the third
   at the bike competition.
3. Andrew did better than Corey at the swimming contest,
   but Corey was better than Andrew at the bike contest.
4. Corey was never the last.
5. Dan won the bike contest,
   but Sandy was better than him at the swimming contest.
What was the winning order in the swimming and bike contest?
?- start,false.
Swimming contest=[sandy,andrew,corey,dan]
Bike contest= [dan,corey,sandy,andrew]
**************************************************/
start:- sol(S,B),write('Swimming contest='),write(S),nl,
    write('Bike contest= '),write(B),nl.

sol(Swim,Bike):- St=[sandy,andrew,corey,dan],
        permutation(Swim,St),permutation(Bike,St),
    Swim=[Hs|_],andrew\==Hs,Bike=[Hb|_],andrew\==Hb,     %% 1
    Bike=[_,_,Hs,_],                                     %% 2
    before(andrew,corey,Swim),before(corey,andrew,Bike), %% 3
    not(last(Swim,corey)),not(last(Bike,corey)),         %% 4
    Bike=[dan|_],before(sandy,dan,Swim).                 %% 5

/*  ?-before(a,c,[m,a,v,c,d]).
    true
```

```
before (X,Y,List) checks if X is before Y in the List.
Starts from Left to right (normal order..).
*/
before(X,Y,L):- append(_,[X|R],L),member(Y,R).
/*       File: Puzzles/ifthen.pro      Author: If Then (sol MM)
    Title: The false Statements
1. Only one of these statements is false
2. Only two of these statements is false
3. Only three of these statements is false
4. Only four of these statements is false
5. All five of these statements is false
Which if any is true?
?- start.
I = [false,false,false,true,false] ;
*/
start:- find(I),write(I).
/* mem/2: tests/generates if all the elements in the first list are
members in the second list.
| ?- mem([X,Y,Z],[a,b]).
X = Y = Z = a ;
X = Y = a ,Z = b ;
X = Z = a ,Y = b ;
X = a ,Y = Z = b ;
X = b ,Y = Z = a ;
X = Z = b ,Y = a ;
X = Y = b ,Z = a ;
X = Y = Z = b ;       There are 2^3 = 8 solutions.
*/
mem([],L).
mem([H|T],L):- member(H,L),mem(T,L).
```

```
find([A1,A2,A3,A4,A5]) :-  mem([A1,A2,A3,A4,A5],[true,false]),
                           Sol=[A1,A2,A3,A4,A5],
                           count(false,Sol,N),
         ((A1-> N=1 ) ; (not A1 -> not N=1)),
         ((A2-> N=2 ) ; (not A2 -> not N=2)),
         ((A3-> N=3 ) ; (not A3 -> not N=3)),
         ((A4-> N=4 ) ; (not A4 -> not N=4)),
         ((A5-> N=5 ) ; (not A5 -> not N=5)).

/* counts how many times an element (A) occurs in a list (first level)
?- count(b,[a,b,c,b],N).
N=2
*/
count(A,[],0).
count(A,[A|T],N):- count(A,T,N1),N is N1+1,!.
count(A,[_|T],N):- count(A,T,N).
```

```
/* File: Puzzles/queen2.pro  also see: Bratko's textbook
Title: Eight Queens problem
Is it possible to place eight queens on an empty chessboard so
that no queen is attacking any other, i.e., no two queens are in the same row,
the same column, or along the same diagonal?
    ?- start(S),write(S),false.
    [1/4,2/2,3/7,4/3,5/6,6/8,7/5,8/1]
    [1/4,2/7,3/5,4/2,5/6,6/1,7/3,8/8]
    ..
Comments (MM): this is a way of generating Alternate Sign Matrices-ASM
that is matrices which have the sum 1 on each row and each column, but only that subset which
does not include  (-1).
Their number is known and easy to find. The number of the ASM of (n x n)was an open problem
till '95.
**************************************************/
start(S):- S=[1/Y1,2/Y2,3/Y3,4/Y4,5/Y5,6/Y6,7/Y7,8/Y8],
           permutation([Y1,Y2,Y3,Y4,Y5,Y6,Y7,Y8],[1,2,3,4,5,6,7,8]),
           list_comb(2,S,L),
           all_prop(not_diagonal,L).

not_diagonal([X/Y,X1/Y1])- A is abs(X-X1),B is abs(Y-Y1),A \= B.

/* mem1(Lr,L). For comb/3. Does not generate [a,b] and [b,a].
    ?- mem1([X,Y],[a,b,c]).
    [a,b][a,c][b,c]
*/
mem1([],Y).
mem1([H|T],Y):-member(H,Y),rest(H,Y,New),mem1(T,New).
/* rest(A,L,R). Returns the rest of the list after the first occurrence of A.
    | ?- rest(a,[a,b,c,d],I). I = [b,c,d]
    | ?- rest(a,[b,c,d],I).       I = []
```

```
*/
rest(A,L,R):-Y=[A|R],append(X,Y,L),!.
/* comb(N,L,Res). Combinations. Arrangements without " order".
    | ?- comb(2,[a,b,c],I).
    I = [a,b] ;  I = [a,c] ;  I = [b,c] ;
*/
comb(N,L,X):-length(X,N),mem1(X,L).
/* list_comb(N,L,Res).
    ?-  list_comb(2,[a,b,c,d],L).
    L = [[a,b],[a,c],[a,d],[b,c],[b,d],[c,d]]
*/
list_comb(N,L,Res):- findall(X,comb(N,L,X),Res).
%%all_prop(Prop,L) - checks if all elements in a list have a certain property
all_prop(P,L):-forall(member(X,L),(F=..[P,X],call(F))).
/*
| ?- findall(S,start(S),L),write(L),length(L,N).
[1 / 1,2 / 5,3 / 8,4 / 6,5 / 3,6 / 7,7 / 2,8 / 4]
[1 / 1,2 / 6,3 / 8,4 / 3,5 / 7,6 / 4,7 / 2,8 / 5]
[1 / 1,2 / 7,3 / 4,4 / 6,5 / 8,6 / 2,7 / 5,8 / 3]
....
[1 / 7,2 / 5,3 / 3,4 / 1,5 / 6,6 / 8,7 / 2,8 / 4]
[1 / 8,2 / 2,3 / 4,4 / 1,5 / 7,6 / 5,7 / 3,8 / 6]
[1 / 8,2 / 2,3 / 5,4 / 3,5 / 1,6 / 7,7 / 4,8 / 6]
[1 / 8,2 / 3,3 / 1,4 / 6,5 / 2,6 / 5,7 / 7,8 / 4]
[1 / 8,2 / 4,3 / 1,4 / 3,5 / 6,6 / 2,7 / 7,8 / 5]
N = 92
*/
```

```
/* File: Puzzle/knight.pro
Title:  Knight's Tour 3x3 board        Luger: page 650.
Can the chess piece called horse/knight move around an empty
chessboard and touch each of the 3 x 3 squares once and only once?
The knight makes L-shaped moves (over two in one direction and
then over one in a perpendicular  direction).
Write a program to find out and display the path.
The knight starts from the up-left corner.
        -----------------
        1   |   2   |   3
        -----------------
        4   |   5   |   6
        -----------------
        7   |   8   |   9
-----------------------------------------------------------*/
move(1,6).   move(1,8). move(2,9).move(2,7).
move(3,4).   move(3,8). move(4,9).move(4,3).
move(6,7).   move(6,1). move(7,6).move(7,2).
move(8,1).   move(8,3). move(9,2).move(9,4).
/*
Solution 1. If there is a path says yes.
    ?- path(1,9).
    true.
    ?- path(1,5).
    ERROR: Out of local stack
path(Z,Z).
path(X,Y):- move(X,W),path(W,Y).
-----------------------------------------------------------*/
/* Solution 2.   Keep track of visited nodes using a predicate
called: visited(?Node)
    ?- path(1,9).
```

```
     Yes
     ?- path(1,5).
     No
:- dynamic visited/1.
path(Z,Z).
path(X,Y):- move(X,W),not(visited(W)),assert(visited(W)),path(W,Y).

start:- retractall(visited(_)),path(1,9).
-----------------------------------------------------------*/
/*Solution 3. Build the path in a list L
?- path(1,9,[1]).
true
?- path(1,5,[1]).
false
path(Z,Z,L).
path(X,Y,L):- move(X,W),not(member(W,L)),path(W,Y,[W|L]).
-----------------------------------------------------------*/
/*Solution 4.  Build and display the path in a list L
?- path(1,5,[1],L).
false
?- path(1,9,[1],L).
L = [9, 7, 6, 1] ;
L = [9, 2, 7, 6, 1] ;
L = [9, 4, 3, 8, 1] ;
L = [9, 2, 7, 6, 1] ;
L = [9, 4, 3, 8, 1] ;

path(Z,Z,L,L).
path(X,Y,L,R):- move(X,W),not(member(W,L)),path(W,Y,[W|L],R).
-----------------------------------------------------------*/
/*Graph (nodes and arcs): path in a graph
```

```
We interpret the move(Node1,Node2) as arc in a graph with Nodes 1-9.
Our problem is to find a path between to given Nodes S,
F: initial(S) and final(F)
All the possible paths (from 1 to 9):
    ?-start,false.
    [9,2,7,6,1]
    [9,4,3,8,1]
-------------------------------------------------------*/
%% Finding a path in graph used for finding a path from 1 to 9
initial(1).      %% 1 is initial node
final(9).            %% 9 is final node
%% Starting Node is Start, Sol is a list of Nodes
%% beginning with Start ending with the final Node.
start:- initial(Start),path(Start,[Start],Sol),
        reverse(Sol,Res),write(Res).

path(Node,Path,[Node|Path]):- final(Node).
path(Node,Path,Sol):- move(Node,N1),not(member(N1,Path)),
                 path(N1,[Node|Path],Sol).
```

```
/* File: Puzzles/seating.pro
   Title: Now seating
Suppose 2 boys and 3 girls go to the theater and they all sit on the same row.
The row has only 5 seats.
1. Generate all different ways they can sit in such a way that the boys
are at the end of the row. Count them.
2. Generate all different ways they can sit in such a way that girls
are at the end of the row. Count them.
3. Generate all different ways they can sit in such a way that boys sit
between the girls.
?- findall(S,start1(S),L),length(L,N),write(N).
L = [[b1, g1, g2, g3, b2], [b1, g2, g1, g3, b2], ...]
N = 12
?- findall(S,start2(S),L),length(L,N),write(N).
L = [[g1, g3, b1, b2, g2], [g1, g3, b2, b1, g2], [g1, b1, g3, b2, g2], ...]
N = 24
*************************************************************/
start1(Sol):- Sol=[X,Y,Z,U,V],permutation([X,V],[b1,b2]),
              permutation(Sol,[b1,b2,g1,g2,g3]).

start2(Sol):- Sol=[X,Y,Z,U,V],member(X,[g1,g2,g3]),
                 member(V,[g1,g2,g3]),permutation(Sol,[b1,b2,g1,g2,g3]).

start3(Sol):- Sol=[X,Y,Z,U,V],permutation([X,Z,V],[g1,g2,g3]),
                 permutation([Y,U],[b1,b2]).
```

```
/* File: Puzzles/magic.pro
Author: MM+David Grello- AI-class 2002
Title: Magic Square.
Generate all the possible 3 x 3 magic squares with numbers from digits from 1 to 9 (the digits
occur just once) (the sum on each line and column is the same).
Uses SWI predicate: permutation/2, sumlist/2
?- findall(X,(X=[A,B,C,D,E,F,G,H,I],square(X)),length(L,N).
L = [[1,5,9,6,7,2,8,3,4],[1,5,9,8,3,4,6,7,2],... ]
N = 72
There are 72 solutions. All have the same sum=15
?- start,fail.
Magic Square 3x3
    5-9-1
    3-4-8
    7-2-6
Magic Square 3x3
    5-1-9
    3-8-4
    7-6-2
Magic Square 3x3
    9-5-1
    4-3-8
    2-7-6
**********************************************************/
start:- S=[A,B,C,D,E,F,G,H,I],
        permutation([1,2,3,4,5,6,7,8,9],S),
        sumlist([A,B,C],N), sumlist([D,E,F],N),
        sumlist([G,H,I],N),
        sumlist([A,D,G],N), sumlist([B,E,H],N),
        sumlist([C,F,I],N),write_square(S).
```

```
write_square([A,B,C,D,E,F,G,H,I]):- write(A-B-C),nl,
                write(D-E-F),nl,
                write(G-H-I),nl,nl.
```

```
/*  File: Puzzles/buffalo.pro
Title: Babysitting
Author: Scott Marley http://brownbuffalo.sourceforge.net
Publication: Dell Logic Puzzles April, 1998    Page: 7 Stars: 1
Each weekday, Bonnie takes care of five of the neighbors' children. The children's names are
Keith, Libby, Margo, Nora, and Otto; last names are Fell, Gant, Hall, Ivey, and Jule. Each
is a different number of years old, from two to six. Can you find each child's full name and
age?
1. One child is named Libby Jule.
2. Keith is one year older than the Ivey child,
   who is one year older than Nora.
3. The Fell child is three years older than Margo.
4. Otto is twice as many years old as the Hall child.
USES SWI:member/2, permutation/2
?- start(S),write_list(S),fail.
[keith,fell,5]
[libby,jule,6]
[margo,hall,2]
[nora,grant,3]
[otto,ivey,4]
*****************************************/
start(Sol):- F=[keith,libby,margo,nora,otto],
             L= [fell,grant,hall,ivey,jule],
          Sol=[[F1,L1,A1],[F2,L2,A2],[F3,L3,A3],[F4,L4,A4],[F5,L5,A5]],
          F=[F1,F2,F3,F4,F5],    % if order is not important!
          member([libby,jule,_],Sol),  % 1
          permutation([L1,L2,L3,L4,L5],L), % write correspondence
          permutation([A1,A2,A3,A4,A5],[2,3,4,5,6]),
member([keith,_,AgeK],Sol),member([_,ivey,AgeI],Sol),AgeK is AgeI+1,% 2
          member([nora,_,AgeN],Sol),AgeI is AgeN+1,
          member([margo,_,AgeM],Sol),
```

```
        member([_,fell,AgeF],Sol),AgeF is 3+AgeM,
        member([otto,_,AgeO],Sol),member([_,hall,AgeH],Sol).

/* There are two types of restrictions:
a) positive ones- when you fill the Solution
b) negative ones where you need the space to be instantiated already:
not X doesn't have sense unless you know from where to take X
! always put a) before b)
*/
```

```
/* File: Puzzles/gradepuzzle.pl
Author: sol M.Malita
From: http://cis2000.bizland.com/puzzles07.htm
Five friends in the sixth form took the same combination of A - level subjects.
Each obtained a different grade in each subject taken, and no two students had the same grade
in the same subject. From the information given below, can you work out the full details
of each student's results?
- Andrew outscored Bridget in Physics, and Neil in Math.
- Wendy was the only girl to get a "C" grade, but she managed no "A" grades..
- The pupil with an "E" in Math gained a "B" in Chemistry, but was not awarded a "C" in
Physics.
- Paul's Physics grade was a "D" and his highest grade was a "C".
- The "B" in Math did not go to the same student as the "E" in Physics.
- Bridget's best result was in Chemistry, but her Math grade was lower than Paul's
What are the obtained grade for each subject by each student?
-------------------------------------------------------------
Solution.
Andrew had A in Math, C in Chemistry and B in Physics.
Bridget had D in Math, A in Chemistry and E in Physics.
Neil had E in Math, B in Chemistry and A in Physics.
Paul had C in Math, E in Chemistry and D in Physics.
Wendy had B in Math, D in Chemistry and C in Physics.
Solution in Prolog.
| ?- start(I).
[[andrew,a,c,b],[bridget,d,a,e],[neil,e,b,a],[paul,c,e,d],[wendy,b,d,c]]
*************************************************************/
/* S= [[Name,Math,Chemistry,Physics],[Name,Math,Chemistry,Physics],..] */
grades([a,b,c,d,e]).
start(S):-

S=[[andrew,Am,Ac,Ap],[bridget,Bm,Bc,Bp],[neil,Nm,Nc,Np],[paul,Pm,Pc,Pp],[wendy,Wm,Wc,Wp]],
```

```
    Pp=d, % Paul's Physics grade was a "D"
% no two students had the same grade in the same subject
    permutation([Ac,Nc,Pc,Bc,Wc],[a,b,c,d,e]),
    permutation([Am,Nm,Pm,Bm,Wm],[a,b,c,d,e]),
    permutation([Ap,Np,Pp,Bp,Wp],[a,b,c,d,e]),
% Each obtained a different grade in each subject taken
    is_set([Am,Ac,Ap]),is_set([Nm,Nc,Np]),
    is_set([Pm,Pc,Pp]),is_set([Bm,Bc,Bp]),
    is_set([Wm,Wc,Wp]),
    first([Pm,Pc,Pp],c), %  and his(Paul) highest grade was a "C"
% Andrew outscored Bridget in Physics, and Neil in Math
    first([Ap,Bp],Ap),best([Am,Nm],Am),
% Wendy was the only girl to get a "C" grade, but she managed no "A" grades
    not(member(c,[Bm,Bc,Bp])),    % Brigdet (was the other girl) and she has no C
    not(member(a,[Wm,Wc,Wp])),    % Wendy has no A
    first([Bm,Bc,Bp],Bc),         % Bridget's best result was in Chemistry
    first([Pm,Bm],Pm),            % but her Math grade was lower than Paul's
% The "B" in Math did not go to the same student as the "E" in Physics
    member([_,b,_,Y],S),Y\==e,
    member([_,e,b,Gradep],S), % The pupil with an "E" in Math gained a "B" in Chemistry,
    Gradep\==c. % but was not awarded a "C" in Physics
/* first/2 finds the best grade
?-first([b,c,d,a],R).
R=a
*/
first(L,F):- sort(L,R),R=[F|_].
```

```
/*   File: Puzzle/jazz.pro     (solution MM)
    Title: Sounds Logical?
    From: Dell- Math Puzzles Logic Problems,   Jan 2002 p.15
" Giant Book of Challenging Thinking Puzzles",
Michael A. DiSPezio,Sterling Pub. House, ISBN 1-4027-1090-9, 2003
Saturdays and Sundays are the days that Sheila, Ramon and Niko shop together for music. The
CD's they purchase are either rock or jazz. When they visit the music store, each person will
purchase one and only one CD. Here are the rules that givern their selections:
    1. Either Sheila or Ramon will pick rock, but not both of them.
    2. If Sheila picks rock, Niko picks jazz.
    3. Niko and Ramon do not both pick jazz.
    Which one of the three purchased a jazz CD on Saturday and a Rock CD on Sunday.

S1= sheila buys on Saturday  S2= sheila buys on Sunday
R1= ramon buys on Saturday  R2= ramon buys on Sunday
N1= niko buys on Saturday  N2= niko buys on Sunday
Solution = [S1,R1,N1,S2,R2,N2]
?- start.
     SATURDAY           SUNDAY
Sheila Ramon Niko Sheila Ramon Niko
jazz  rock  jazz  jazz  rock  jazz
************************************************/
start:- find(Sol), mywrite(Sol),nl.
mywrite(L) :- write('   SATURDAY           SUNDAY       '),nl,
          write('Sheila Ramon Niko Sheila Ramon Niko'),nl,
              forall(member(X,L),(write(X),write('  '))).
/*
find(Sol):- Sol =  [S1,R1,N1,S2,R2,N2],
        member(S1,[jazz,rock]),
        member(S2,[jazz,rock]),
        member(N1,[jazz,rock]),
```

```
        member(N2,[jazz,rock]),
        member(R1,[jazz,rock]),
        member(R2,[jazz,rock]),
        is_set([S1,R1]), %% 2. If Sheila picks rock, Niko picks jazz.
        is_set([S2,R2]), %% that is different
    not([N1,R1]=[jazz,jazz]), %% 3. Niko and Ramon do not both pick jazz.
    not([N2,R2]=[jazz,jazz]),
        (S1=jazz ; (S1=rock,N1=jazz) ) , %% if sheila picks jazz does not matter
        (S2=jazz ; (S2=rock,N2=jazz)).   %% %% is sheila picks rock, Nick has jazz
*/
/* Fill an empty list with elements from another list
?- mem([X,Y,Z],[a,b]).
X = a Y = a Z = a ;
X = a Y = a Z = b ;
X = a Y = b Z = a ;
X = a Y = b Z = b ;
X = b Y = a Z = a ;
X = b Y = a Z = b ;
X = b Y = b Z = a ;
X = b Y = b Z = b ;
************************************/
mem([],Y).
mem([H|T],Y):-member(H,Y),mem(T,Y).

find(Sol):- Sol =  [S1,R1,N1,S2,R2,N2],
        mem(Sol,[jazz,rock]),
        is_set([S1,R1]), %% 2. If Sheila picks rock, Niko picks jazz.
        is_set([S2,R2]), %% that is different
         not([N1,R1]=[jazz,jazz]), %% 3. Niko and Ramon do not both pick jazz
         not([N2,R2]=[jazz,jazz]),
        (S1=jazz ; (S1=rock,N1=jazz) ) , %% is sheila picks jazz does not matter
```

```
(S2=jazz ; (S2=rock,N2=jazz)).   %% %% is sheila picks rock, Nick has jazz
```

```
/* File: flip.pro
   Author: Mark Herod - Student 2005
   Title: Flipping Coins
Place 3 coins with their indicated side facing up as shown.
In 3 moves arrange the coins so that all three have the same side facing down.
A move counts of flipping 2 coins over to their opposite side.
        H   T   H
Note: (Always different configuration)
Flipping the pair of outer coins 3 times doesn't count!
    state(s) = node is graph
    [head, tail, head] = [h,t,h]
    ?- start,fail.
    [h, t, h]
    [t, h, h]
    [h, h, t]
    [t, t, t]

    [h, t, h]
    [h, h, t]
    [t, h, h]
    [t, t, t]
*********************************************/
start:- initial(S),path(S,[],Sol),
        reverse(Sol,Res),length(Sol,4),
        prettyprint(Res).

%% at the beginning All are on the same bank
    %initial([h,t,h]).
    initial([h,h,h]).
%%
    %%final([h,h,h]).
```

```
    final([t,t,t]).
path(Node,Path,[Node|Path]):- final(Node).
path(Node,Path,Sol):- move(Node,N1),not(member(N1,Path)),
                      path(N1,[Node|Path],Sol).
%%
    opp(h,t).
    opp(t,h).
prettyprint(L):- forall(member(X,L),(write(X),nl)).
move([A,B,C],[A1,B1,C]):- opp(A,A1),opp(B,B1).  % first two coins
move([A,B,C],[A,B1,C1]):- opp(C,C1),opp(B,B1).  % last two coins
```

```
/* File: Puzzles/anagram.pro
   Title: Generates all anagrams of given a word
?-start.
Write your name? ale.
here are the anagrams:
ale     lae     lea     ael     eal     ela

Uses SWI predicates: permutation/2, name/2
?- name(mel,I).
I = [109, 101, 108]
?- name('Turing',L).
L = [84, 117, 114, 105, 110, 103]
?- name(X,[100,101,102]).
X = def
*********************************************************/
start:-write('Write your name? '),read(X),
        write('\nhere are the anagrams:\n'),
        name(X,L),permutation(L,R),name(Word,R),
        write(Word),tab(5).
```

/* File: Puzzles/message.pro
Author: M. Malita
Title: Message sending

Email is out of order at St. Mary's College and the teacher wants to tell Robert something urgent.
The teacher meets Craig and asks him to tell Robert she wants to speak with him.
Craig says that if he meets Robert its OK, but else he will send the message to everyone he meets and the message will go further.
Each student tells each student he meets that the teacher waits for Robert in her office.
The students meet each other (we don't know in what order):

1) Craig meets John and Jason
2) Jason meets Kiki and Adam and David
3) Adam meets Scott and Jeremy
4) Jeremy meets John and Scott
5) Kiki meets Chris
6) Chris meets David and Adam
7) David meets Robert

Do you think the teacher will wait for ever in her office until Robert comes?
Display all the possible paths from Craig to Robert.
Ex:- ?- start,nl,false.

```
[ro,da,ch,ki,ja,ad,je,jo,cr]
[ro,da,ja,ad,je,jo,cr]
[ro,da,ja,ki,ch,ad,je,jo,cr]
[ro,da,ch,ad,je,jo,cr]
[ro,da,ch,ki,ja,ad,sc,je,jo,cr]
[ro,da,ja,ad,sc,je,jo,cr]
[ro,da,ja,ki,ch,ad,sc,je,jo,cr]
[ro,da,ch,ad,sc,je,jo,cr]
[ro,da,ch,ki,ja,cr]
[ro,da,ch,ad,ja,cr]
[ro,da,ja,cr]
```

```
*/
arc(cr,jo).  arc(cr,ja).
arc(ja,ki).  arc(ja,ad).  arc(ja,da).
arc(ad,sc).  arc(ad,je).
arc(je,jo).  arc(je,sc).
arc(ki,ch).
arc(ch,da).  arc(ch,ad).
arc(da,ro).
arc1(Y,X):- arc(X,Y);arc(Y,X).

initial(cr). final(ro).
start:- initial(S),bkt(S,[],Sol),write(Sol).

bkt(Node,Path,[Node|Path]):- final(Node).
bkt(Node,Path,Sol):- arc1(Node,N1), not member(N1,Path),
                     bkt(N1,[Node|Path],Sol).
```

```
/*  File: Puzzles/phone.pro
Author: Mihaela Malita
Title: Transforms phone numbers written with digits in words.
*/
/* a -97  e-101  i- 105 o-111 u-117 y-121  ?- two_vowels(soap).
    vowel(V):-member(V,[97,101,105,111,117,121]).
    two_vowels(Word):-name(Word,Lw),append(_,[V1,V2|_],Lw),vowel(V1),vowel(V2).
*/
    key(1,[1]).
    key(2,[a,b,c]).
    key(3,[d,e,f]).
    key(4,[g,h,i]).
    key(5,[j,k,l]).
    key(6,[m,n,o]).
    key(7,[p,r,s]).
    key(8,[t,u,v]).
    key(9,[w,x,y]).
    key(0,[0]).

    phone([],[]).
    phone([N|T],[X|R]):-key(N,Letters),member(X,Letters),phone(T,R).
/* same as
possible(D,Let):-key(D,L),member(Let,L).
?- possible(2,F).
F = a ;
F = b ;
F = c.
?- maplist(possible,[1,2,3],R).
R = [1, a, d] ; R = [1, a, e] ; R = [1, a, f] ; R = [1, b, d] ;
R = [1, b, e] ; R = [1, b, f] ; R = [1, c, d] ; R = [1, c, e] ;
R = [1, c, f].
```

```
phone2(P,R):- findall(S,maplist(possible,[1,2,3],S),R).
**********************************************************/
/*
start:-write('your phone as a list(ex:[4,3,6,6]? '),read(Phone),
        (phone(Phone,L),write(L),nl,false).
*/
    vowel(a).    vowel(e).    vowel(i).    vowel(o).    vowel(u).    vowel(y).

/* has_vowels([m,a,r,y]),2).  */
    has_vowels([],0).
    has_vowels([H|T],N):- vowel(H),has_vowels(T,K),N is K+1,!.
    has_vowels([_|T],N):- has_vowels(T,N).

    start:-write('Your phone as a list (ex:[4,3,6,6]='),read(Phone),
         (phone(Phone,L),(has_vowels(L,2),write(L),nl ; true),false).

/*  Discussion. How many possiblities there are?
    Improve program using name/2.
*/
```

```
/*  File: Puzzles/flashlight2.pro
Solution: M.Malita
Title: Four Men Crossing a Bridge (from Microsoft interview process)
There are four men who would all like to cross a rickety old bridge.
The old bridge will only support 2 men at a time, and it is night time, so every crossing
must use the one flashlight that they all share.
The four men each have different walking speeds; the fastest each of them can cross is 1
minute, 2 minutes, 5 minutes, and 10 minutes.
If they pair up, since they must share the flashlight, they can only cross in the time that
it would take the slower of the two.
Given that the shortest time to get them all across is 17 minutes total, how should they all
cross?
*/
/*
We describe the problem as Nodes in a graph and the solution means to find a path from
the initial node to the final node.
assume the names of the four people are: a,b,c,d
state = node is graph
state = [Time,Flash_place,[a,b,c,d],[]]
Bank can be left (l) or right (r).  Thus Flash_place is l or r.
[5,l,[a,b,c],[d]] - means 5 minutes passed and a,b,c are on the left bank and d is on the
right
| ?- start,fail.
Found sol=[17,r,[],[a,b,c,d]]
[15,l,[a,b],[c,d]]
[14,r,[b],[c,d,a]]
[4,l,[b,c,d],[a]]
[2,r,[c,d],[a,b]]
[0,l,[a,b,c,d],[]]
no
*/
```

```
start:- initial(S),path(S,[],Sol),write('Found sol='),forall(member(X,Sol),(write(X),nl)).
/* finding a path in a graph from initial node to final node */
path(N,P,[N|P]):- final(N).
path(N,P,Sol):- arc(N,N1),not(member(N1,P)),path(N1,[N|P],Sol).
/* at the beginning All are on the same bank and Time=0 */
initial([0,l,[a,b,c,d],[]]).
/* at the end they have all to be on the other bank and Time=17*/
final([17,r,[],[a,b,c,d]]).
/* opposite bank. */
opp(l,r).  opp(r,l).
/* time for crossing the bridge - time is a system predicate */
tim(a,1).
tim(b,2).
tim(c,5).
tim(d,10).
/* define the arcs (or move conditions from a state node) to another state(node) */
arc([T1,F1,L1,R1], [T2,F2,L2,R2]):- opp(F1,F2),
         ((F1=l,cross(X,L1),
         take(X,L1,L2),append(X,R1,R2),findtime(X,T),T2 is T1+T);
     (F1=r,cross(X,R1),
         take(X,R1,R2),append(X,L1,L2),findtime(X,T),T2 is T1+T)),T2 < 18.

/* remove all elements in S from L result is in R */
take(S,L,R):- findall(Z,(member(Z,L),not(member(Z,S))),R).

/* we know just one or two persons cross the bridge */
findtime([X],Tim):- tim(X,Tim),!.
findtime([A,B],Tim):- tim(A,Ta),tim(B,Tb),Tim is max(Ta,Tb),!.

/* take all the combinations of 1 person, and 2 persons from our group: [a,b,c,d] */
cross(X,L):- comb(1,L,X); comb(2,L,X).
```

```
/* mem1(Lr,L). For comb/3. Same as mem/2 but does not generate [a,b] and [b,a].
     ?- mem1([X,Y],[a,b,c]).
     [a,b][a,c][b,c]
*/
mem1([],Y).
mem1([H|T],Y):-member(H,Y),rest(H,Y,New),mem1(T,New).

rest(A,L,R):- append(_,[A|R],L),!.
/* comb(N,L,Res). Combinations. Arrangements without " order".
     | ?- comb(2,[a,b,c],I).
     I = [a,b] ;  I = [a,c] ;  I = [b,c] ;
*/
comb(N,L,X):-length(X,N),mem1(X,L).
```

```
/*File: Puzzles/canibmm.pl
Author: (sol) MM
Title: Missionaries and cannibals
On a river bank there are 3 cannibals and 3 missionaries.
Here is a boat with 2 places for crossing the river.
If on a bank there remain more cannibals than missionaries
then it is dangerous, since the cannibals may eat them.
How could they all cross the river without being in danger?
Representation:
    Banks are labeled East(e) and West(w).
    A state (node) is:
    [Boats_Place,#Miss_east,#Cann_east,#Miss_west,#Cann_west]
or short:
[e,Me,Ce,Mw,Cw] - the boat is on the bank labeled e
         on the e bank there are Me missionaries and Ce cannibals
     on the w bank there are Mw missionaries and Cw cannibals
[w,2,0,1,3] - means: the boat is on the bank w
              on bank e: Me=2 Ce=0
          on bank w: Mw=1 ( 1  missionary) Cw=3 (3 cannibals)
              (this is a BAD state).
    ?-   start.
    Boat Me Ce Mw Cw
    [e, 3, 3, 0, 0]
    [w, 2, 2, 1, 1]
    [e, 3, 2, 0, 1]
    [w, 3, 0, 0, 3]
    [e, 3, 1, 0, 2]
    [w, 1, 1, 2, 2]
    [e, 2, 2, 1, 1]
    [w, 0, 2, 3, 1]
    [e, 0, 3, 3, 0]
```

```
    [w, 0, 1, 3, 2]
    [e, 0, 2, 3, 1]
    [w, 0, 0, 3, 3]
*************************************************************/
%% declare the initial state (node) and final state(node)
initial([e,3,3,0,0]).
final([w,0,0,3,3]).

/* Explain the states (nodes) prohibited by my problem,
Bad nodes should be avoided when you search for the path.
*/
bad([_,M1,C1,M2,C2]):- (M1 > 0,M1 < C1);(M2 > 0,M2 < C2).

/*  The problem is reduced to the pattern:
    find a path in a directed graph.
    The following predicates are almost the same as in graph theory.
       We added the condition (not(bad(State) ).
*/
start:- initial(S),path(S,[],Sol),reverse(Sol,Res),my_write(Res).

path(Node,Path,[Node|Path]):- final(Node).
path(Node,Path,Sol):- move(Node,N1),
              not(bad(N1)),
              not(member(N1,Path)),
                     path(N1,[Node|Path],Sol).
/* The art of solving the problem is to describe the arcs(edges).
   After you decided the representation for the states.
*************************************************************/
/* Find a possible configuration for the boat.
   Take from one bank the configuration,
   add the configuration to the other bank
```

```
*/
move([e, Me1, Ce1, Mw1, Cw1], [w, Me2, Ce2, Mw2, Cw2]):-
        boat(M,C), %% how many Missionaries and Cannibals are in the boat
        Me1 >= M,Ce1 >= C, %% check if it is a good proposal
        Me2 is Me1 - M,   %% take them from one shore
        Ce2 is Ce1 - C,
        Mw2 is Mw1 + M,   %% put them on another shore
        Cw2 is Cw1 + C.

move([w, Me1, Ce1, Mw1, Cw1], [e, Me2, Ce2, Mw2, Cw2]):-
        boat(M,C),
        Mw1 >= M,Cw1 >= C,
        Me2 is Me1 + M,
        Ce2 is Ce1 + C,
        Mw2 is Mw1 - M,
        Cw2 is Cw1 - C.
/* boat/2 is a generator. M and C are the numbers of missionaries and cannibals
   which can sit in the boat. Their total number must be <= 2.
    ?- boat(M,C).
    M=0     C=1
    M=1     C=0 ...
*/
boat(M,C):- member([M,C],[[0,1],[1,0],[1,1],[2,0],[0,2]]).

/* Write each element of a list on a separate line */
my_write(L):- write('Boat Me Ce Mw Cw'),nl,forall(member(X,L),(write(X),nl)).
```

```
/* File: Puzzles/goat.pro
   Title: The goat problem
A farmer has to cross a river with a wolf, a goat and a cabbage.
He has a boat, but in the boat he can take just one thing.
He cannot let the goat alone with the wolf or the goat with the cabbage.
It's obvious why. What is the solution?
We describe the problem as Nodes in a graph and the solution means to find a path from
the initial node to the final node.
    state(s) = node is graph
    s(farmer(Bank),goat(Bank),cabbage(Bank),wolf(Bank)).
    Bank can be East (e) or West (w).
*****************************************/
start:- initial(S),path(S,[],Sol),
        reverse(Sol,Res),prettyprint(Res).
%% at the beginning All are on the same bank
    initial(s(farmer(e),goat(e),cabbage(e),wolf(e))).
%% at the end they have all to be on the other bank
    final(s(farmer(w),goat(w),cabbage(w),wolf(w))).

path(Node,Path,[Node|Path]):- final(Node).
path(Node,Path,Sol):- move(Node,N1), not(bad(N1)),
                      not(member(N1,Path)),
                      path(N1,[Node|Path],Sol).

%% opposite bank East is opposite to west Bank. opp/2
    opp(e,w).    opp(w,e).

/* a bad state (s) is a state to be avoided. In this case:
Goat cannot be left alone with wolf or cabbage
The bank where the farmer is (T) has to be opposite
Where the goat is together with the wolf or the cabbage
```

```
T, G, C , W are variables for the banks (e or w)
*************************************************************/
bad(s(farmer(T),goat(G),cabbage(C),wolf(W))):-
                  (G=W,opp(T,G)); (G=C,opp(T,G)).

%% Farmer takes goat
    move(s(farmer(T),goat(T),cabbage(C),wolf(W)),
             s(farmer(T1),goat(T1),cabbage(C),wolf(W))):-opp(T,T1).

%% Farmer takes cabbage
    move(s(farmer(T),goat(G),cabbage(T),wolf(W)),
        s(farmer(T1),goat(G),cabbage(T1),wolf(W))):-opp(T,T1).

%% Farmer takes wolf
    move(s(farmer(T),goat(G),cabbage(C),wolf(T)),
             s(farmer(T1),goat(G),cabbage(C),wolf(T1))):-opp(T,T1).

%%  Farmer goes alone
    move(s(farmer(T),goat(G),cabbage(C),wolf(W)),
             s(farmer(T1),goat(G),cabbage(C),wolf(W))):-opp(T,T1).

prettyprint(L):- forall(member(X,L),(write(X),nl)).
```

```
/*   File: Puzzles/river.pro
     Author(sol): M. Malita IMPROVE!
     Problem: Cross River
There are 4 adults who want to cross a river.
They come upon a boy and a girl playing in a rowboat.
The boat can hold either two children or one adult.
Can the adults succeed in crossing the river? If so, how?
     ?- start.
     Bank   AE CE AW CW
     %% Bank Adult-East-Bank Children-East-Bank Adult-West-Bank Children-West-Bank
     [east, 4, 2, 0, 0]
     [west, 4, 0, 0, 2]
     [east, 4, 1, 0, 1]
     [west, 3, 1, 1, 1]
     [east, 3, 2, 1, 0]
     [west, 3, 0, 1, 2]
     [east, 3, 1, 1, 1]
     [west, 2, 1, 2, 1]
     [east, 2, 2, 2, 0]
     [west, 2, 0, 2, 2]
     [east, 2, 1, 2, 1]
     [west, 1, 1, 3, 1]
     [east, 1, 2, 3, 0]
     [west, 1, 0, 3, 2]
     [east, 1, 1, 3, 1]
     [west, 0, 1, 4, 1]
     [east, 0, 2, 4, 0]
     [west, 0, 0, 4, 2]
*************************************************/
initial([east,4,2,0,0]).  final([west,0,0,4,2]).
start:-      initial(S),path(S, [], Sol),
```

```
        reverse(Sol,Res),mywrite(Res).
boat(A,K):- member([A, K], [[1,0],[0,1],[0,2]]).
move([east, Ae1, Ke1, Aw1, Kw1], [west, Ae2, Ke2, Aw2, Kw2]):-
        boat(A,K),
        Ae1>=A,Ke1>=K,
        Ae2 is Ae1 - A,
        Ke2 is Ke1 - K,
        Aw2 is Aw1 + A,
        Kw2 is Kw1 + K.
move([west, Ae1, Ke1, Aw1, Kw1], [east, Ae2, Ke2, Aw2, Kw2]):-
        boat(A,K),
        Aw1 >= A,Kw1 >= K,
        Ae2 is Ae1 + A,
        Ke2 is Ke1 + K,
        Aw2 is Aw1 - A,
        Kw2 is Kw1 - K.

path(N, Path, [N|Path]):-  final(N).
path(N, Path, Sol):- move(N, NI),not(member(NI, Path)),path(NI, [N|Path], Sol).

mywrite(L):- write('Bank AE CE AW CW'),nl,forall(member(X,L),(write(X),nl)).
```

```
/* file: Puzzles/moremoney.pro Cryptarithmetic.
   SEND+
   MORE
   -----
   MONEY
Solution is: [S,E,N,D,M,O,R,Y]=[9, 5, 6, 7, 1, 0, 8, 2],
Assign different values 0-9 to all letters S,E,N,D,M,O,R,Y in such a way that the equation
holds
**************************/
start(Sol) :- Sol = [S,E,N,D,M,O,R,Y],
    between(1,9,S), between(0,9,E),between(0,9,N),between(0,9,D),
    between(1,9,M), between(0,9,O),between(0,9,R),between(0,9,Y),
    is_set(Sol),   %% works but very slow!
    SendMore is 1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E,
    Money is 10000*M + 1000*O + 100*N + 10*E + Y,
    SendMore = Money,
    write('Sol=').
/******************** TODO***********************
start(Sol) :-    Sol = [S,E,N,D,M,O,R,Y],
    %% numlist(0,9,Ln), % Ln
    %%%mysubset([S,E,N,D,M,O,R,Y,_],Ln),
    S \== 0, M \== 0, %% cannot start with 0
    SendMore is 1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E,
    Money is 10000*M + 1000*O + 100*N + 10*E + Y,
    SendMore = Money,
    write('Sol=').
*****************************************/
```

```
/*  File: Puzzles/words.pro (Sol): Mihaela Malita IMPROVE!
    Title:  For finding word squares    3x3
    Generates word squares.
A word square is an arrangement of letters in a square such that each row and each column is
a word in the dictionary.
    ?- start.
    Solution for wordsquare
    t-a-b
    e-p-a
    n-e-t
Algorithm:
0) Fix your vocabulary (lexicon).
1) Write down your square with variables:
        A-B-C
         D-E-F
         G-H-I
2) Put condition: all row and columns are words:
   For rows:
    word([A,B,C]).  word([D,E.F]).  word([G,H,I]).
   For colums:
    word([A,D,G]).  word([B,E.H]).  word([C,F,I]).
3) Put together all the variables you look for in one predicate.
        wordsq(A,B,C,D,E,F,G,H,I).
4) Write start. Look for all solutions.
************************************************/
/*       data base with words */
word([t,a,b]).   word([e,p,a]).
word([b,a,t]).   word([t,e,n]).
word([t,a,n]).   word([r,a,t]).
word([r,a,n]).   word([a,t,e]).
word([j,a,b]).   word([j,o,e]).
```

```
word([b,e,t]).   word([a,p,e]).
word([n,e,t]).   word([a,p,t]).
word([o,f,t]).   word([o,p,t]).

wordsq(A,B,C,D,E,F,G,H,I):-   word([A,B,C]),word([D,E,F]),
                        word([G,H,I]),word([A,D,G]),
                        word([B,E,H]),word([C,F,I]).
/* Just one solution
start:- write('Solution for wordsquare\n'),
    wordsq(A,B,C,D,E,F,G,H,I),
        write(A-B-C),nl,
        write(D-E-F),nl,
    write(G-H-I).
-------------------------------------------*/
/* All solutions   */
start:-  write('Solution for wordsquare\n'),
         forall(wordsq(A,B,C,D,E,F,G,H,I),
             (write('\nWord Square:\n'),
             write(A-B-C),nl,
         write(D-E-F),nl,
         write(G-H-I))).
```

```
/* File: Puzzles/coins0.pro Author: Mihaela Malita
Give us sum and transform it in coins. US coins are:
1 Quater = 25 cents
1 Dime   = 10 cents
1 Niquel = 5 cents
1 Penny  = 1 cent

?-   start.
Transform in coins. What sum? 86.
S=86   Q=3   D=1   N=0   P=1
Continue(y/n)? y.
Transform in coins. What sum? 99.
S=99   Q=3   D=2   N=0   P=4
Continue(y/n)? n.
************************************************/
coins(S):- possible([S,Q,D,N,P]),
           write('Sum='),write(S),tab(3),
           write('Q='),write(Q),tab(3),
           write('D='),write(D),tab(3),
           write('N='),write(N),tab(3),
           write('P='),write(P),tab(3).

possible([S,Q,D,N,P]):-
         between(0,4,P),
         between(0,19,N),
         between(0,9,D),
         between(0,4,Q),
        S is Q * 25 + D * 10 + N * 5 + P.

start:- write('Transform in coins. What sum?'),read(S),coins(S),yes_or_no.
```

```
yes_or_no:-write('Continue(y/n)? '),read(X),(X=n; X=no; X=nu; X=q ).
```

```
/*File: Puzzles/sudokuSolve.pro Author: Mihaela Malita
Title:  SUDOKU Solver. From: http://www.sudoku.com/
A sort of Magic Square 9 x9
all integers: 1-9 fill on lines and columns (no repeats)
also blocks (9) do not repeat
SWI predicates: permutation/2  and is_set/1
*************************************************************/
%% start:- write('Enter known positions?'),read(S),solve(S).

start:-S=[[_, _, _, 3, 4, 2, _, _, _],
      [5, 4, _, _, 7, _, _, 8, _],
      [_, _, 2, _, _, 5, 4, _, 6],
      [_, 6, _, 2, _, _, _, _, _],
      [3, _, 8, _, _, _, 2, _, 4],
      [_, _, _, _, _, 8, _, 7, _],
      [6, _, 9, 1, _, _, 5, _, _],
      [_, 3, _, _, 8, _, _, 1, 9],
      [_, _, _, 5, 3, 9, _, _, _]],
         solve(S).

write_square(S):- nl,forall(member(X,S),(write(X),nl)),nl.
/*
two lines are different if they do not have any elements on the same position.
?- different([a,b],[b,a]).
true
?- different([a,b],[a,a]).
false
---------------------------------*/
different([],[]).
different([H1|T1],[H2|T2]):- not(var(H1)),not(var(H2)),
                             not(H1=H2),different(T1,T2),!.
```

```
different([H1|T1],[H2|T2]):- different(T1,T2),!.

/*
Integers do not repeat in blocks 3x3
-----------------------------------*/
good([A1,A2,A3,A4,A5,A6,A7,A8,A9],[B1,B2,B3,B4,B5,B6,B7,B8,B9]):-
    notvars([A1,A2,A3,B1,B2,B3],G1),
    notvars([A4,A5,A6,B4,B5,B6],G2),
    notvars([A7,A8,A9,B7,B8,B9],G3),
    is_set(G1),is_set(G2),is_set(G3).

/*
Collect non variables from lines:
?- notvars([_,2,_,_,5,_,7,_,9,1],L).
L = [2, 5, 7, 9, 1]
-----------------------------------*/
notvars(L,G):-findall(X,(member(X,L),not(var(X))),G).

solve(S):-  Nine=[1,2,3,4,5,6,7,8,9],
        S=[L1,L2,L3,L4,L5,L6,L7,L8,L9],
        write_square(S),
        permutation(L1,Nine),   %% Line 1
        %%write_square(S),
        good(L1,L2),good(L1,L3),
        different(L1,L4),different(L1,L5),
        different(L1,L6),different(L1,L7),
        different(L1,L9),different(L1,L9),
    write_square(S),
        permutation(L2,Nine),   %% Line 2
        good(L1,L2),good(L2,L3),
    different(L2,L4),different(L2,L5),different(L2,L6),
```

```
different(L2,L7),different(L2,L8),different(L2,L9),
write_square(S),
    permutation(L3,Nine),   %% Line 3
    good(L3,L1),good(L3,L2),
different(L3,L4),different(L3,L5),different(L3,L6),
different(L3,L7),different(L3,L8),different(L3,L9),
write_square(S),
permutation(L4,Nine),   %% Line 4
    different(L4,L1),different(L4,L2),different(L4,L3),
different(L4,L5),different(L4,L6),different(L4,L7),
    different(L4,L8),different(L4,L9),
write_square(S),
permutation(L5,Nine),   %% Line 5
    good(L5,L4),
    different(L5,L1),different(L5,L2),different(L5,L3),
different(L5,L6),different(L5,L7),
    different(L5,L8),different(L5,L9),
write_square(S),
permutation(L6,Nine),   %% Line 6
good(L6,L4),good(L6,L5),
    different(L6,L1),different(L6,L2),different(L6,L3),
    different(L6,L7),different(L6,L8),different(L6,L9),
write_square(S),
    permutation(L7,Nine),   %% Line 6
different(L7,L1),different(L7,L2),different(L7,L3),
    different(L7,L4),different(L7,L5),different(L7,L6),
    different(L7,L8),different(L7,L9),
write_square(S),
permutation(L8,Nine),   %% Line 6
good(L8,L7),good(L8,L9),
    different(L8,L1),different(L8,L2),different(L8,L3),
```

```
different(L8,L4),different(L8,L5),different(L8,L6),
 write_square(S).
```

```
/* File: Puzzles/disorder.pro Solution: Mihaela Malita
Title: Disorder.
Rearrange the numbers in the figure in such a way that
consecutive numbers do not TOUCH each other.
They cannot align side by side, ups and downs or diagonally.
    1               A
2   3   4       B   C   D
5   6   7       E   F   G
    8               H

?- start,false.
     7
4    1     5
6    8     3
     2
*****************************************************/
start:-  S=[A,B,C,D,E,F,G,H],permutation([1,2,3,4,5,6,7,8],S),
         notconsec(A,B),notconsec(A,C),notconsec(A,D),
        notconsec(B,C),notconsec(B,E),notconsec(B,F),
        notconsec(C,E),notconsec(C,F),notconsec(C,D),notconsec(C,G),
         notconsec(D,G),notconsec(D,F),notconsec(E,F),notconsec(E,H),
         notconsec(F,H),notconsec(F,G),
         write_square(S),nl.

notconsec(X,Y):- not(X is Y+1 ; X is Y-1).

write_square([A,B,C,D,E,F,G,H]):-
                 tab(4),write(A),nl,
                write(B-C-D),nl,
                write(E-F-G),nl,
                 tab(4), write(H),nl.
```

```
/* File: Puzzles/secretNumber.pro
Author: Mihaela Malita
Title:  Problem 60. Secrete Number I. Giant book of Mensa Mind Challenges
Sort of MASTERMIND.
You have to guess a SECRET = 5 digit integer (0-9) no repeats.
You know that the SECRET has in common with the following numbers (R (Right)
means one digit in common different place, B(Bingo) means one digit in common in the same
place)
8951 - RR
2169 - RB
3694 - RB
4721 - RB
1237 - RRR
?- start(I).
[3,7,1,9]
**********************************************/
start(S):-  S=[X,Y,Z,W],   %% to check S=[3,7,1,9],
            mem(S,[1,2,3,4,5,6,7,8,9]),is_set(S),
            intersection([8,9,5,1],S,L1),length(L1,2), % 2 elements in common
            intersection([2,1,6,9],S,L2),length(L2,2),samePlace(S,[2,1,6,9],[_]),
            % 2 elements in common and 1 same position
            intersection([3,6,9,4],S,L3),length(L3,2),samePlace(S,[3,6,9,4],[_]),
            intersection([4,7,2,1],S,L4),length(L4,2),samePlace(S,[4,7,2,1],[_]),
            intersection([1,2,3,7],S,L5),length(L5,3), % 3 elements in common
           write(S).

/* Find elements the same in the same position
?- samePlace([a,b],[b,a],R).
R=[]
?- samePlace([a,b],[a,a],R).
false.
```

```
-----------------------------------------------------*/
samePlace([],[],[]).
samePlace([H|T1],[H|T2],[H|R]):- samePlace(T1,T2,R).
samePlace([_|T1],[_|T2],R):- samePlace(T1,T2,R).

mem([],L).
mem([H|T],L):-member(H,L),mem(T,L).
```

```
/* File: Puzzles/soccer.pro
88. giant book of Mensa Mind challenges 2003 isbn 1-4027-1049-6
Five soccer teams United, County, Rovers Albion and Thistle took part in a league tournament.
Their colors were white,yellow,green and red blue. No teams were tied in the standings at the
end. From the following information determine for each team its captain, colors,
and position in which it finished in the league.
1. Rovers did not win the league, but finished higher than the 4th
2. Neither Albion nor the team in green finished in the top three.
3. Evans captained the team in yellow.
4. Cooke's team finished a head of County which was captained by Dixon.
5. Allen's team finished second and Boyle's team finished last
6. the team in white finished lower than both united and the team in blue but above Evan's
team.
7. Albion was not the green team and United was not the blue team.
solution Team=[team,captain,color]
S=[[united,cooke,red],[rovers,allen,blue],[county,dixon,white],[albion,evans,yellow],[thist
le,boyle,green]]
**********************************************************************/
start:- S=[T1,T2,T3,T4,T5],
          T1=[H1,N1,C1], T2=[H2,allen,C2], T3=[H3,N3,C3], T4=[H4,N4,C4], T5=[H5,boyle,C5],
          Colors=[yellow,red,blue,white,green],
           Teams=[united,albion,rovers,thistle,county],
           permutation([C1,C2,C3,C4,C5],Colors),
           permutation([H1,H2,H3,H4,H5],Teams),
            H1 \== rovers, before([rovers,_,_],T4,S),  %1
            member([_,evans,yellow],S),        %2
             ([albion,_,_]=T4 ; [albion,_,_]=T5 ),
             ([_,_,green]=T4 ; [_,_,green]=T5 ), %2
             member([albion,_,Ca],S),Ca \== green,
             member([united,_,Cu],S),Cu \== blue, %7
              before([_,cooke,_],[county,dixon,_],S),
```

```
            before([united,_,_],[_,_,white],S), %6
            before([_,_,blue],[_,_,white],S),   %6
            before([_,_,white],[_,evans,_],S),  %6
             forall(member(T,S),(write(T),nl)). %6

before(X,Y,L):-append(_,[X|T],L),member(Y,T).
```

```
/* File: Puzzles/arch.pro
Arch Friends  from http://brownbuffalo.sourceforge.net  Author: Mark T. Zegarelli
Publication: Dell Logic Puzzles, Issue: April, 1998, Page: 7 Stars: 1
Harriet, upon returning from the mall, is happily describing her four shoe purchases to her
friend Aurora.
Aurora just loves the four different kinds of shoes that Harriet bought (ecru espadrilles,
fuchsia flats, purple pumps, and suede sandals), but Harriet can't recall at which different
store
(Foot Farm, Heels in a Handcart, The Shoe Palace, or Tootsies) she got each pair.
Can you help these two figure out the order in which Harriet bought each pair of shoes, and
where she bought each?
1. Harriet bought fuchsia flats at Heels in a Handcart.
2. The store she visited just after buying her purple pumps was not Tootsies.
3. The Foot Farm was Harriet's second stop.
4. Two stops after leaving The Shoe Place, Harriet bought her suede sandals.
Determine: Order - Shoes - Store
?-start(X).
S=[[pumps,shoePlace],[espadrilles,footFarm],[sandals,tooties],[flats,heelsHandcart]].
Solution by % © Copyright Doug Edmunds 2000
First, purple pumps, The Shoe Place
Second, ecru espadrilles, Foot Farm
Third, suede sandals, Tooties
Fourth, fuchsia flats, Heels in a Handcart
**************************************************/
start(S):- S=[[H1,T1],[H2,T2],[H3,T3],[H4,T4]],
       Shoes=[pumps,espadrilles,sandals,flats],
       Stores=[shoePlace,footFarm,tooties,heelsHandcart],
       permutation([H1,H2,H3,H4],Shoes),
       permutation([T1,T2,T3,T4],Stores),
       member([flats,heelsHandcart],S),       %% 1. Harriet bought fuchsia flats at Heels
in a Handcart.
```

```
       nth1(2,S,[_,footFarm]),                  %% 3. The Foot Farm was Harriet's second stop.
       nth1(N1,S,[pumps,_]),                 %% 2. The store she visited just after buying her
purple pumps was not Tootsies.
       nth1(N2,S,[_,tooties]), not(N2 is N1+1),
       nth1(N3,S,[_,shoePlace]),   %% 4. Two stops after leaving The Shoe Place, Harriet
bought her suede sandals
       nth1(N4,S,[sandals,_]), N4 is N3 + 2.
```

```
/* File: Puzzles/bibmm.pro
Author: MMalita
Work in Progress!! Library for Logic Puzzles
    SWI: append/3 member/2 intersection/3  permutations/2   flatten/2 last/2 length/2
numlist(1,N,L) sumlist/2
    all/3
    all_prop(P,L): if all in a list have Property P/1
    arrange/3
    comb/3
    first/2  first([H|_],H).  same as nth0(0,L,H). count/3
    count_list_all/3
    count_list_comb/3
    count_list_arange/3
    count_prop/3
    intersection_n/3
    generate_sol/2
    list_comb/3
    list_all/3
    list_arrange/3
    mem/2 (=subset)       SWI: subset/2
    mem1/2
    SWI: nth0/3
    place/3          SWI: nth1(?Index, ?List, ?Elem)
    remove/3         SWI: select(E,L,R)
    rest/3
    next/3
    SWI: nextto(X,Y,L).
    before/3         SWI: nextto(X,Y,L).
    neighbor/5       diagonal/1   not_diagonal/1
    set/1  (=no_duplicates,=alldifferent)  SWI: is_set/1
    set_equal/2          SWI: permutation/2
```

```
    sum/2                SWI: sumlist/2
    write_list/1
%   first(List,First).
first(List,H):- List=[H|_].
/* generate_sol(S,N). Generate solution list - size N */
generate_sol(S,N):- length(S,N).
/* write_list(L). Format predicates.*/
write_list(L):-forall(member(X,L),(write(X),nl)).
/* count(A,L,N). Counts occurrences
    ?- count(a,[b,a,c,d,a],N).
    N = 2
*/
count(_,[],0).
count(A,[A|L],N):- count(A,L,N1), N is N1+1,!.
count(A,[_|L],N):- count(A,L,N).
/* count_prop(P,L,N). Count elements with a certain property
    ?-count_prop(integer,[1,2,a,b,3,4],N).
    N=4
    ?- count_prop(atomic,[b,[a,c],d,a],N).
    N = 3
*/
count_prop(P,[],0).
count_prop(P,[A|L],N):- F=..[P,A],call(F),count_prop(P,L,N1),N is N1+1,!.
count_prop(P,[_|L],N):- count_prop(P,L,N).
/* all_prop(P,L). All elements from a list (first level) have property Prop. EX: check if
all are integres in a list
?- P=integer,L=[1,2,3],forall(member(X,L),(F=..[P,X],call(F))).
P = integer,
L = [1, 2, 3].
*/
all_prop(P,[]).
```

```
all_prop(P,[H|T]):- F=..[P,H],call(F),all_prop(P,T),!.
/* same as:
all_prop(P,L):-forall(member(X,L),(F=..[P,X],call(F))).
*/
/* mem(Lr,L). Elements from Lr are all members in L.
    ?- mem([X,Y],[a,b,c]).
    X = Y = a ;  X = a ,Y = b ;   X = a ,Y = c ;   X = b ,Y = a ;
    X = Y = b ;  X = b ,Y = c ;   X = c ,Y = a ;   X = c ,Y = b ;
    X = Y = c ;
*/
mem([],Y).
mem([H|T],Y):-member(H,Y),mem(T,Y).
/* all(N,L,X). All possible N-length lists with elements from [a,b,c,d]:
    | ?- all(2,[a,b,c],I).
    I = [a,a] ;  I = [a,b] ;  I = [a,c] ;  I = [b,a] ;
    I = [b,b] ;  I = [b,c] ;  I = [c,a] ;  I = [c,b] ;  I = [c,c] ;
*/
all(N,L,X):-length(X,N),mem(X,L).
/* list_all(N,L,R).
    ?- list_all(2,[a,b,c],R).
    R = [[a,a],[a,b],[a,c],[b,a],[b,b],[b,c],[c,a],[c,b],[c,c]]
*/
list_all(N,L,Res):-findall(X,all(N,L,X),Res).
/* count_list_all(N,L,M).   k list has N = number is N^K
    ?- count_list_all(2,[a,b,c,d],Many).
    Many = 16
*/
count_list_all(N,L,Many):-findall(X,all(N,L,X),Res),length(Res,Many).
/* arrange(N,L,R). Lists have to be sets! That is: elements do not repeat.
    ?- arrange(2,[a,b,c],R).
    R = [a,b] ;  R = [a,c] ;  R = [b,a] ;  R = [b,c] ;  R = [c,a] ;  R = [c,b] ;
```

```
*/
arrange(N,L,X):-length(X,N),mem(X,L),is_set(X).
no_duplicates(M):- is_set(M).
alldifferent(M):-  is_set(L).
/* list_arrange(N,L,Res).
    ?- list_arange(2,[a,b,c,d],R).
    R = [[a,b],[a,c],[a,d],[b,a],[b,c],[b,d],[c,a],[c,b],[c,d],[d,a],[d,b],[d,c]]
*/
list_arrange(N,L,Res):- findall(X,arrange(N,L,X),Res).
/* count_list_arrange/3. A(k,n)= n!/(n-k)!
    ?- count_list_arrange(2,[a,b,c,d],R).
    R = 12
*/
count_list_arrange(N,L,Many):- list_arrange(N,L,Res),length(Res,Many).
list_permutations(L,R):-findall(X,permutations(L,X),Res).
/* count_list_permutations(L,N). N=length(L)!*/
count_list_permutations(L,N):-list_permutations(L,R),length(R,N).

/* mem1(Lr,L). For comb/3. Same as mem/2 but does not generate [a,b] and [b,a].
    ?- mem1([X,Y],[a,b,c]),write([X,Y]),false.
    [a,b][a,c][b,a][b,c][c,a][c,b]no
*/
mem1([],Y).
mem1([H|T],Y):-member(H,Y),rest(H,Y,New),mem1(T,New).
/* rest(A,L,R). For comb/3. Returns the rest of the list after the first occurrence of A.
    | ?- rest(a,[a,b,c,d],I). I = [b,c,d]
    | ?- rest(a,[b,c,a,d],I). I = [d]
    | ?- rest(a,[b,c,d],I).       I = []
rest(X,[],[]):-!.
rest(X,[X|T],T):-!.
rest(X,[_|T],R):-rest(X,T,R).
```

```
*/
%%same as
rest(A,L,R):-Y=[A|R],append(X,Y,L),!.
/* comb(N,L,Res). Combinations. Arrangements without " order".
    | ?- comb(2,[a,b,c],I).
    I = [a,b] ;  I = [a,c] ;  I = [b,c] ;
*/
comb(N,L,X):-length(X,N),mem1(X,L).
/* list_comb(N,L,Res).
    ?-  list_comb(2,[a,b,c,d],L).
    L = [[a,b],[a,c],[a,d],[b,c],[b,d],[c,d]]
*/
list_comb(N,L,Res):- findall(X,comb(N,L,X),Res).
/* count_list_comb(N,L,Many). Formula is: C(k,n) = n!/ k! (n-k)!
    or Pascal's formula: C(k,n)=C(k,n-1) + C(k-1,n-1)
    ?-  count_list_comb(2,[a,b,c,d],L).
    L = 6
*/
count_list_comb(N,L,Many):- list_comb(N,L,Res),length(Res,Many).
/* SWI: select/3. Removes the first occurrence of an element.
    ?- select(a,[a,b,a,d,a],I).
    I = [b,a,d,a] ;  I = [a,b,d,a] ; I = [a,b,a,d] ;
    ?- select(X,[a,b,c],I).
    X = a ,  I = [b,c] ;  X = b ,  I = [a,c] ;  X = c ,  I = [a,b] ;
    ?- select(m,[a,b,c,d],I).
    false
    ?- select(m,I,[b]).
    I = [m,b] ;  I = [b,m] ;
We can write remove/3 as follows, with same effect as follows:
    remove(X,[],[]):-!.
    remove(X,[X|T],T):-!.
```

```
    remove(X,[H|T],[H|R]):-myremove(X,T,R).
%%  same as:
    remove(X,L,R):-append(F,[X|R1],L),append(F,R1,R). %skip X
*/
/* Order in a list:List= ..X,Y,.. means:Y is in the right of X
    ?- right(a,b,[c,a,b,m,n]). %% SWI: nextto/3
    true
*/
right(X,Y,L):-   nextto(X,Y,L).
/* next(X,Y,L). If X and Y are next to each other in the list.
    ?- next(X,Y,[a,b,c]).
    X = a ,Y = b ;   X = b ,Y = c ;   X = b ,Y = a ;   X = c ,Y = b ;
    ?- next(a,b,[m,a,b,c]),next(a,c,[m,a,b,c,d]).
    true.
*/
next(X,Y,L):-    nextto(X,Y,L) ; nextto(Y,X,L).
/* neighbor(+X,+Y,X1,Y1,+S). Two cells are neighbors in an array size S
- starts from 0. Assume X and Y are in the range 0 - S.
between(0,I,5) means 0 <= I <= 5
    ?-neighbor(1,1,2,2,3).
    true
    ?-neighbor(1,2,I,J,2).
    I = J = 2 ;  I = 0 ,J = 2 ;
    I = J = 1 ;  I = 2 ,  J = 1 ; I = 0 ,  J = 1 ;
*/
neighbor(X,Y,X1,Y1,S):- ((X1 is X+1, Y=Y1);
             (X1 is X-1, Y=Y1);(X1=X, Y1 is Y+1);
             (X1=X, Y1 is Y-1);(X1 is X+1, Y1 is Y-1);
             (X1 is X-1, Y1 is Y-1);(X1 is X+1, Y1 is Y+1);
             (X1 is X-1, Y1 is Y+1)),between(0,X1,S),
            between(0,Y1,S).
```

```
/*  Chessboard (any SIZE!). Are on a the same diagonal. */
diagonal(X/Y,X1/Y1):-   A is abs(X-X1),A is abs(Y-Y1).
/*  Not on the same diagonal*/
not_diagonal([X/Y,X1/Y1]):- A is abs(X-X1),B is abs(Y-Y1),A \= B.
```